

Gestión de Procesos y Control de Recursos en Linux: Monitoreo, Priorización y Optimización en Entornos Docker

Resumen

La administración eficiente de procesos y recursos es un aspecto fundamental para garantizar el rendimiento y la estabilidad de los sistemas operativos modernos. En este laboratorio se analizaron diferentes mecanismos de monitoreo, generación de carga artificial y control de procesos en un entorno Linux ejecutado mediante Docker. Se realizaron pruebas de saturación de CPU y memoria utilizando la herramienta *stress*, así como experimentos relacionados con prioridades de ejecución mediante *nice* y *renice*, terminación de procesos con señales del sistema y limitación de recursos mediante *cpulimit*. Los resultados obtenidos permitieron observar el comportamiento del sistema bajo diferentes escenarios de carga y comprender la importancia de una adecuada gestión de recursos para optimizar el funcionamiento de aplicaciones y servicios.

Palabras clave

Linux, Docker, procesos, administración de recursos, scheduler, stress, cpulimit, monitoreo de sistemas.

INTRODUCCIÓN

Los sistemas operativos modernos deben administrar eficientemente los recursos computacionales disponibles para garantizar un funcionamiento estable y un rendimiento adecuado de las aplicaciones. Dentro de estas responsabilidades, la gestión de procesos constituye una de las funciones más importantes, ya que permite coordinar la ejecución de múltiples tareas que compiten simultáneamente por el uso del procesador, la memoria y otros recursos del sistema.

Linux proporciona diversas herramientas que facilitan el monitoreo y control de procesos, permitiendo a los administradores analizar el comportamiento del sistema y tomar decisiones orientadas a mejorar su rendimiento. Asimismo, tecnologías como Docker permiten ejecutar aplicaciones dentro de entornos aislados, facilitando la realización de pruebas controladas sin afectar el sistema anfitrión.

En el presente laboratorio se estudiaron diferentes mecanismos de administración de procesos en Linux mediante un conjunto de pruebas prácticas realizadas dentro de un

contenedor Docker. Se analizaron aspectos relacionados con el monitoreo de recursos, la generación de cargas artificiales, la gestión de prioridades, la terminación de procesos y la limitación del uso de CPU. A partir de los resultados obtenidos fue posible comprender el funcionamiento de estas herramientas y su importancia en la administración de sistemas operativos.

I. Preparación del entorno de pruebas

A. Caracterización inicial del sistema

Objetivo de la fase

Antes de realizar las pruebas de carga y monitoreo, fue necesario conocer el estado inicial del entorno de trabajo. Para ello se identificaron los recursos disponibles del sistema, incluyendo la cantidad de núcleos de procesamiento, memoria RAM, procesos activos y estructura jerárquica de los procesos en ejecución.

Esta información permitió establecer una línea base para comparar posteriormente el comportamiento del sistema durante las diferentes pruebas de estrés aplicadas al contenedor.

Herramientas utilizadas

Durante esta fase se utilizaron las siguientes herramientas de administración y monitoreo:

- `htop`
- `free -h`
- `nproc`
- `ps aux`
- `pstree`

Procedimiento realizado

Inicialmente se verificó la cantidad de núcleos disponibles mediante el comando `nproc`. Posteriormente se consultó el estado de la memoria RAM utilizando `free -h`. Para observar los procesos activos y su consumo de recursos se empleó `htop`, mientras que `ps aux` y `pstree` permitieron visualizar la lista de procesos y su organización jerárquica dentro del sistema.

Las capturas obtenidas muestran el comportamiento del sistema en estado de reposo, sin cargas artificiales generadas por herramientas de estrés.

Evidencias observadas

A partir de las capturas se pudo observar que el sistema se encontraba prácticamente inactivo, con la mayoría de los núcleos sin carga significativa y un alto porcentaje de

tiempo de CPU disponible. También se evidenció una cantidad suficiente de memoria RAM libre para la ejecución de las pruebas posteriores.

Entre los procesos identificados se encontraron principalmente los servicios asociados al entorno Linux, procesos del contenedor Docker y terminales Bash utilizadas para la práctica.

Tabla 1. Resultados de las pruebas de saturación de CPU

Comando	Workers	Duración	Núcleos al 100%	CPU por worker
stress --cpu 1 --timeout 30s	1	30 s	1	≈100%
stress --cpu 2 --timeout 30s	2	30 s	2	≈105% – 109%
stress --cpu 4 --timeout 30s	4	30 s	4	≈108% – 109%

Análisis de resultados

Los resultados obtenidos permitieron verificar el comportamiento esperado de la herramienta stress al incrementar progresivamente la cantidad de workers.

Con una única instancia (--cpu 1) se observó la saturación de un solo núcleo, alcanzando aproximadamente el 100% de utilización. Al aumentar a dos workers (--cpu 2), dos núcleos comenzaron a trabajar simultáneamente a máxima capacidad. Finalmente, con cuatro workers (--cpu 4), se registró la saturación de cuatro núcleos de forma concurrente.

Durante todas las pruebas el consumo de memoria permaneció prácticamente estable, indicando que la carga generada estuvo enfocada principalmente en el procesador. Además, el comportamiento observado coincidió con la configuración aplicada en cada experimento, demostrando una relación directa entre el número de workers ejecutados y la cantidad de núcleos utilizados.

Figura 1. Docker para entrar.

```
famahu1785@Famahu:~/docker-lab$ # Volver a entrar  
docker exec -it node-api bash
```

Figura 2. Sistema en reposo.

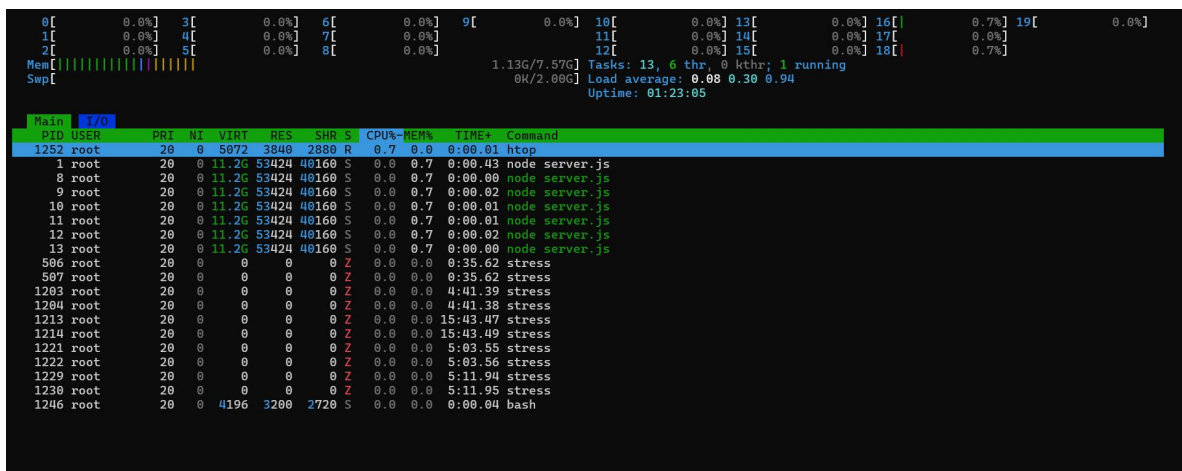


Figura 3. RAM disponible.

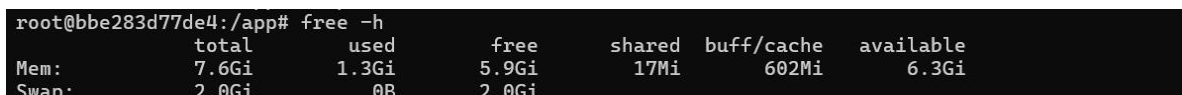


Figura 4. Numero de núcleos.



Figura 5. Lista de procesos en reposo.

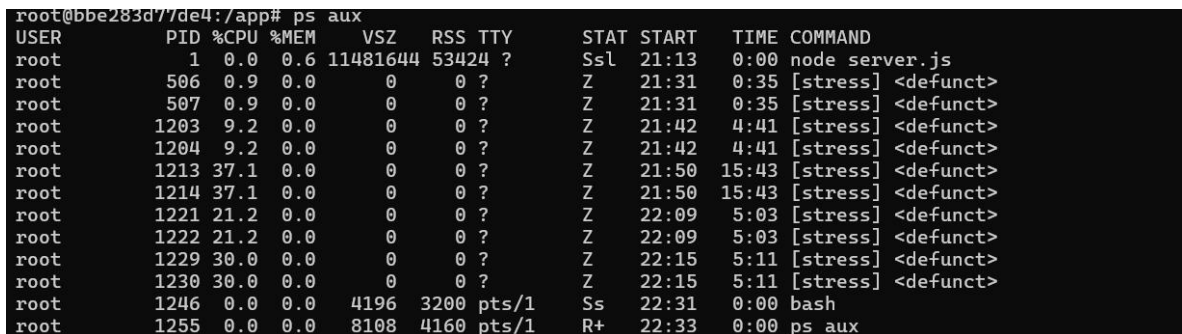


Figura 6. Numero de procesos activos.



Figura 7. Jerarquía de procesos.



Figura 8. Top 5 procesos por CPU.

```
root@bbe283d77de4:/app# ps aux --sort=-%cpu | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1214 35.8  0.0      0     0 ?        Z    21:50   15:43 [stress] <defunct>
root      1213 35.8  0.0      0     0 ?        Z    21:50   15:43 [stress] <defunct>
root      1230 27.6  0.0      0     0 ?        Z    22:15    5:11 [stress] <defunct>
root      1229 27.6  0.0      0     0 ?        Z    22:15    5:11 [stress] <defunct>
```

II. Generación y monitoreo de cargas artificiales

En esta etapa se realizaron diferentes pruebas de estrés sobre el sistema con el objetivo de analizar el comportamiento de los recursos computacionales bajo distintas condiciones de carga. Se evaluó el uso del procesador, memoria RAM y la interacción simultánea de múltiples procesos para observar su impacto sobre el rendimiento general del entorno.

A — Saturación de CPU mediante la herramienta Stress

Objetivo de la fase

El objetivo de esta prueba fue analizar el comportamiento del procesador al incrementar progresivamente la cantidad de workers ejecutados por la herramienta stress. Para ello se realizaron pruebas con 1, 2 y 4 workers de CPU durante intervalos de 30 segundos.

Procedimiento realizado

Se ejecutaron los comandos `stress --cpu 1`, `stress --cpu 2` y `stress --cpu 4`, monitoreando en tiempo real el comportamiento del sistema mediante `htop`. Durante cada ejecución se observó la ocupación de los núcleos y la distribución de la carga generada por los workers.

Tabla 2. Resultados de saturación de CPU

Comando	Workers	Duración	Núcleos al 100%	CPU por worker
<code>stress --cpu 1 --timeout 30s</code>	1	30 s	1	≈100%
<code>stress --cpu 2 --timeout 30s</code>	2	30 s	2	≈105% – 109%
<code>stress --cpu 4 --timeout 30s</code>	4	30 s	4	≈108% – 109%

Análisis de resultados

Los resultados evidenciaron una relación directa entre la cantidad de workers configurados y el número de núcleos utilizados. Con cada incremento de workers aumentó proporcionalmente la cantidad de núcleos saturados, alcanzando niveles cercanos al 100% de utilización.

Las capturas obtenidas en htop muestran claramente cómo las barras de CPU se llenan progresivamente conforme aumenta la carga aplicada.

Figura 9. Terminal 1 — lanza stress de RAM.

```
root@bbe283d77de4:/app# ps aux --sort=-%cpu | head -5
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.6 11481644 53264 ?        Ssl   21:09   0:00 node server.js
root        14  0.0  0.0   4196  2880 pts/0    Ss    21:09   0:00 bash
root       485  0.0  0.0   8108  4160 pts/0    R+    21:17   0:00 ps aux --sort=-%cpu
root       486  0.0  0.0   2392   640 pts/0    R+    21:17   0:00 head -5
root@bbe283d77de4:/app# stress --cpu 1 --timeout 30s
stress: info: [493] dispatching hogs: 1 cpu, 0 io, 0 vm, 0 hdd
```

Figura 10. Terminal 2 — observa mientras corre.

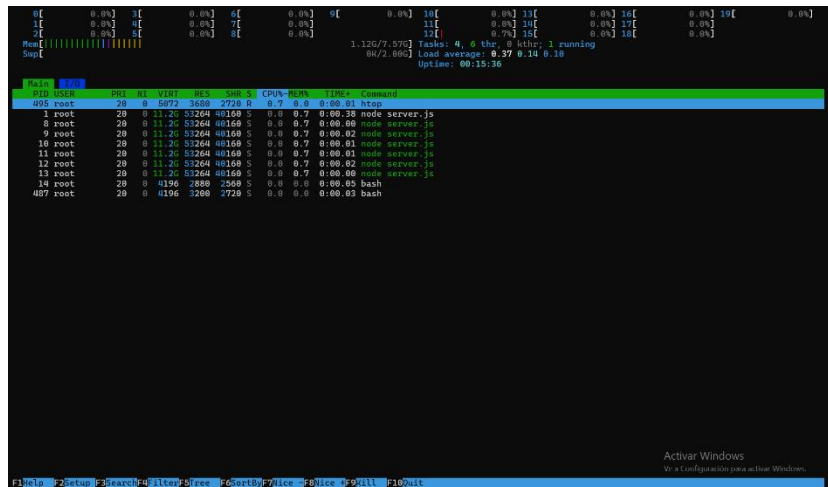


Figura 11. Ahora con 4 CPUs.

```
root@bbe283d77de4:/app# stress --cpu 4 --timeout 30s
stress: info: [500] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [500] successful run completed in 31s
```

Figura 12. Terminal 2 — observa mientras corre.

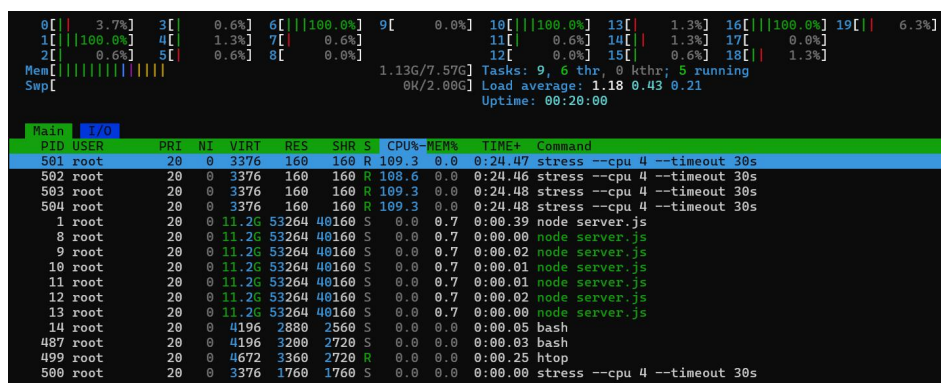


Figura 13. Abrir terminal 2.

```
famahu1785@Famahu:~$ docker exec -it node-api bash
root@bbe283d77de4:/app# stress --cpu 2 --timeout 30s
stress: info: [496] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Figura 14. Terminal 2 — observa en htop.

```
0[ 3.5%] 3[ 0.0%] 6[ 0.5%] 9[ 0.0%] 10[ 6.3%] 13[ 0.0%] 16[ 1.0%] 19[ 2.1%]
1[ 0.0%] 4[ 0.0%] 7[ 0.0%] 11[ 0.0%] 14[ 0.0%] 17[ 0.0%]
2[ 0.5%] 5[ 0.0%] 8[ 0.5%] 12[ 0.5%] 15[ 0.0%] 18[ 0.5%]
Mem[|||||] 1.12G/7.57G Tasks: 4, 6 thr, 0 kthr; 1 running
Swp[ ] 0K/2.00G Load average: 0.95 0.33 0.16
Uptime: 00:16:26

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.38 node server.js
8 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
14 root 20 0 4196 2880 2560 S 0.0 0.0 0:00.05 bash
487 root 20 0 4196 3200 2720 S 0.0 0.0 0:00.03 bash
499 root 20 0 4672 3360 2720 R 0.5 0.0 0:00.01 htop

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice -F8Nice +F9Kill F10Quit
```

B — Saturación de memoria RAM

Objetivo de la fase

Analizar el comportamiento de la memoria principal cuando se generan cargas intensivas mediante la asignación artificial de bloques de memoria utilizando la herramienta stress.

Procedimiento realizado

Se ejecutó el comando correspondiente utilizando workers de memoria configurados para reservar aproximadamente 256 MB cada uno. Paralelamente se monitoreó el consumo de memoria utilizando `watch -n 1 free -h`, observando la evolución de los valores de memoria libre, utilizada y disponible.

Análisis de resultados

Durante la ejecución se observó una disminución progresiva de la memoria libre y un aumento proporcional de la memoria utilizada. Este comportamiento confirmó que los workers estaban reservando correctamente los bloques de memoria solicitados.

Dependiendo del nivel de ocupación alcanzado, el sistema podría comenzar a utilizar espacio de intercambio (swap) para compensar la falta de memoria física disponible.

Figura 15. Terminal 1 — lanza stress de RAM.

```
root@bbe283d77de4:/app# stress --vm 2 --vm-bytes 256M --timeout 60s
stress: info: [524] dispatching hogs: 0 cpu, 0 io, 2 vm, 0 hdd
```

Figura 16. Terminal 2 — observa mientras corre.

```
Every 1.0s: free -h                                     bbe283d77de4: Fri May 29 21:29:03 2026

```

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	1.5Gi	5.7Gi	17Mi	631Mi	6.1Gi
Swap:	2.0Gi	0B	2.0Gi			

Figura 17. Revisión htop.

```
0[ 3.5%] 3[ 0.0%] 6[ 0.5%] 9[ 0.0%] 10[ 6.3%] 13[ 0.0%] 16[ 1.0%] 19[ 2.1%]
1[ 0.0%] 4[ 0.0%] 7[ 0.0%] 11[ 0.0%] 14[ 0.0%] 17[ 0.0%]
2[ 0.5%] 5[ 0.0%] 8[ 0.5%] 12[ 0.5%] 15[ 0.0%] 18[ 0.5%]
Mem[|||||] 1.12G/7.57G Tasks: 4, 6 thr, 0 kthr, 1 running
Swp[|||||] 0K/2.00G Load average: 0.95 0.33 0.16
Uptime: 00:16:26

Main 170
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.38 node server.js
8 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
14 root 20 0 4196 2880 2560 S 0.0 0.0 0:00.05 bash
487 root 20 0 4196 3200 2720 S 0.0 0.0 0:00.03 bash
499 root 20 0 4672 3360 2720 R 0.5 0.0 0:00.01 htop

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit
```

C — Ejecución de un script Python con consumo de CPU y memoria

Objetivo de la fase

Evaluar el impacto de una aplicación desarrollada en Python capaz de consumir simultáneamente recursos de procesamiento y memoria, simulando el comportamiento de una aplicación mal optimizada o con fugas de recursos.

Procedimiento realizado

Se ejecutó un script Python diseñado para crear varios procesos de trabajo. Algunos de ellos realizaban operaciones continuas de procesamiento mediante bucles infinitos, mientras que otros reservaban memoria progresivamente durante la ejecución.

Tabla 3. Consumo registrado por los procesos Python

PID	CPU %
1194	108.5 %
1195	54.6 %
1196	53.3 %
1197	0.7 %

Tabla 4. Recursos utilizados

Recurso	Valor
RAM utilizada	1.19 GB
Load Average	0.18, 0.36, 0.35
Procesos Python	4

Análisis de resultados

Se observó la creación de cuatro procesos asociados al script Python. El proceso con PID 1194 registró el mayor consumo de CPU, alcanzando un valor aproximado de 108.5%.

Además, se evidenció un incremento en el uso de memoria RAM respecto al estado inicial del sistema. Este comportamiento demuestra cómo una aplicación puede consumir simultáneamente distintos recursos del sistema cuando no existe una gestión eficiente de los mismos.

Figura 18. Terminal 1 — crea el script y ejecuta.

```
root@bbe283d77de4:/app# stress --vm 2 --vm-bytes 256M --timeout 60s
stress: info: [524] dispatching hogs: 0 cpu, 0 io, 2 vm, 0 hdd
stress: info: [524] successful run completed in 63s
root@bbe283d77de4:/app# cat > /app/carga.py << 'EOF'
import threading
import time

def consumir_cpu():
    while True:
        pass

def consumir_ram():
    datos = []
    while True:
        datos.append("X" * 10**6)
        time.sleep(0.1)

hilos_cpu = [threading.Thread(target=consumir_cpu) for _ in range(2)]
hilo_ram = threading.Thread(target=consumir_ram)

for h in hilos_cpu:
    h.daemon = True
    h.start()

hilo_ram.daemon = True
hilo_ram.start()

print("Carga iniciada. Esperando 60s...")
time.sleep(60)
print("Finalizado.")
EOF
root@bbe283d77de4:/app# python3 /app/carga.py &
[1] 787
root@bbe283d77de4:/app# Carga iniciada. Esperando 60s...
```

Figura 19. Terminal 2 — observa en htop.

```
0[ 0.0%] 3[ 0.0%] 6[ 0.0%] 9[ 0.0%] 10[ 0.0%] 13[ 0.7%] 16[ 23.8%] 19[ 0.0%]
1[ 0.0%] 4[ 0.0%] 7[ 0.0%] 11[ 0.0%] 14[ 26.2%] 17[ 0.0%]
2[ 0.0%] 5[ 49.0%] 8[ 0.0%] 12[ 0.0%] 15[ 0.0%] 18[ 0.0%]
Mem[|||||] 1.19G/7.57G Tasks: 7, 9 thr, 0 kthr; 2 running
Swp[ 0K/2.00G] Load average: 0.18 0.36 0.35
Uptime: 00:31:14
```

Main	T/O	PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
		1194	root	20	0	288M	68320	5280	S	108.5	0.9	0:07.55	python3 /app/carga.py
		1195	root	20	0	288M	68320	5280	S	54.6	0.9	0:03.77	python3 /app/carga.py
		1196	root	20	0	288M	68320	5280	R	53.3	0.9	0:03.70	python3 /app/carga.py
		1197	root	20	0	288M	68320	5280	S	0.7	0.9	0:00.04	python3 /app/carga.py
		1	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.39	node server.js
		8	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.00	node server.js
		9	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.02	node server.js
		10	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.01	node server.js
		11	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.01	node server.js
		12	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.02	node server.js
		13	root	20	0	11.2G	53264	40160	S	0.0	0.7	0:00.00	node server.js
		14	root	20	0	4712	3360	2560	S	0.0	0.0	0:00.06	bash
		487	root	20	0	4196	3200	2720	S	0.0	0.0	0:00.03	bash
		506	root	20	0	0	0	0	Z	0.0	0.0	0:35.62	stress
		507	root	20	0	0	0	0	Z	0.0	0.0	0:35.62	stress
		1198	root	20	0	4672	3520	2880	R	0.0	0.0	0:00.00	htop

Figura 20. Matar el script.

```
hilo_ram.daemon = True
hilo_ram.start()

print("Carga iniciada. Esperando 60s...")
time.sleep(60)
print("Finalizado.")
EOF
root@bbe283d77de4:/app# python3 /app/carga.py &
[1] 787
root@bbe283d77de4:/app# Carga iniciada. Esperando 60s...
Finalizado.

[1]+  Done                  python3 /app/carga.py
root@bbe283d77de4:/app# python3 /app/carga.py &
[1] 1194
root@bbe283d77de4:/app# Carga iniciada. Esperando 60s...
pkill -f carga.py
root@bbe283d77de4:/app# |
```

D — Competencia entre múltiples procesos

Objetivo de la fase

Simular un escenario similar al de un servidor en producción, donde diferentes procesos compiten simultáneamente por los recursos disponibles del sistema.

Procedimiento realizado

Se ejecutaron de forma concurrente tres tipos de carga:

- stress para consumo intensivo de CPU.
- Un proceso Python con bucle infinito.
- Un proceso dd generando actividad continua.

Posteriormente se monitoreó el comportamiento general del sistema mediante htop.

Análisis de resultados

La ejecución simultánea de múltiples procesos provocó un incremento considerable en la utilización del procesador y en el valor del Load Average. En las capturas se observan varios procesos ejecutándose con porcentajes elevados de CPU al mismo tiempo y múltiples núcleos trabajando de forma concurrente.

Asimismo, comenzaron a aparecer procesos en estado Zombie, evidenciando problemas asociados a la finalización y gestión de algunos procesos hijos durante las pruebas de carga.

Este escenario permitió observar cómo la competencia por recursos puede afectar el rendimiento general del sistema cuando no existe ningún mecanismo de control o limitación.

Figura 21. Terminal 1 — lanza los 3 procesos.

```
root@bbe283d77de4:/app# stress --cpu 2 &
python3 -c "while True: pass" &
dd if=/dev/zero of=/dev/null bs=1M &
[2] 1200
[1] Terminated python3 /app/carga.py
[3] 1201
[4] 1202
root@bbe283d77de4:/app# stress: info: [1200] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Figura 22 y 23. Terminal 2 — observa el caos.

```
0[||||100.0%] 3[||||100.0%] 6[| 0.7%] 9[| 1.3%] 10[| 0.7%] 13[| 0.7%] 16[||||100.0%] 19[| 0.0%]
1[| 2.0%] 4[| 0.7%] 7[||||100.0%] 11[| 0.0%] 14[| 1.3%] 17[| 0.0%]
2[| 1.3%] 5[| 0.7%] 8[| 0.7%] 12[| 0.7%] 15[| 0.0%] 18[| 0.0%]
Mem[|||||] 1.14G/7.57G Tasks: 11, 6 thr, 0 kthr, 6 running
Swp[| 0K/2.00G] Load average: 0.76 0.45 0.39
Uptime: 00:33:52

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%-MEM% TIME+ Command
1201 root 20 0 13840 7520 4960 R 109.2 0.1 0:11.11 python3 -c while True: pass
1202 root 20 0 3572 2400 1280 R 109.2 0.0 0:11.10 dd if=/dev/zero of=/dev/null bs=1M
1203 root 20 0 3376 320 320 R 109.2 0.0 0:11.10 stress --cpu 2
1204 root 20 0 3376 320 320 R 109.2 0.0 0:11.10 stress --cpu 2
1 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.40 node server.js
8 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
14 root 20 0 4712 3360 2560 S 0.0 0.0 0:00.06 bash
487 root 20 0 4196 3200 2720 S 0.0 0.0 0:00.03 bash
506 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
507 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
1200 root 20 0 3376 1760 1760 S 0.0 0.0 0:00.00 stress --cpu 2
1205 root 20 0 4672 3520 2880 R 0.0 0.0 0:00.00 htop
```

```
0[||||100.0%] 3[||||100.0%] 6[| 0.0%] 9[| 0.0%] 10[| 0.0%] 13[| 0.0%] 16[| 0.0%] 19[| 0.0%]
1[| 1.3%] 4[| 0.0%] 7[||||100.0%] 11[| 0.0%] 14[| 0.0%] 17[||||100.0%]
2[| 0.0%] 5[| 0.0%] 8[| 0.0%] 12[| 0.0%] 15[| 0.0%] 18[| 0.0%]
Mem[|||||] 1.13G/7.57G Tasks: 11, 6 thr, 0 kthr, 5 running
Swp[| 0K/2.00G] Load average: 3.07 1.24 0.67
Uptime: 00:35:11

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%-MEM% TIME+ Command
1201 root 20 0 13840 7520 4960 R 104.5 0.1 1:33.97 python3 -c while True: pass
1202 root 20 0 3572 2400 1280 R 104.5 0.0 1:33.96 dd if=/dev/zero of=/dev/null bs=1M
1203 root 20 0 3376 320 320 R 105.1 0.0 1:33.97 stress --cpu 2
1204 root 20 0 3376 320 320 R 105.1 0.0 1:33.96 stress --cpu 2
1 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.40 node server.js
8 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
14 root 20 0 4712 3360 2560 S 0.0 0.0 0:00.06 bash
487 root 20 0 4196 3200 2720 S 0.0 0.0 0:00.03 bash
506 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
507 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
1200 root 20 0 3376 1760 1760 S 0.0 0.0 0:00.00 stress --cpu 2
1205 root 20 0 4672 3520 2880 R 0.0 0.0 0:00.05 htop
```

III. Gestión y control de procesos

En esta etapa se realizaron diferentes pruebas orientadas a la administración de procesos dentro del sistema Linux. Se estudiaron los mecanismos de terminación de procesos, el manejo de prioridades mediante el scheduler del kernel y las técnicas de limitación de recursos para controlar aplicaciones con alto consumo de CPU.

A — Terminación de procesos mediante señales

Objetivo de la fase

Analizar el comportamiento de los procesos al recibir diferentes señales de terminación, comparando la finalización controlada mediante SIGTERM con la terminación forzada mediante SIGKILL.

Procedimiento realizado

Inicialmente se generaron procesos de carga utilizando la herramienta stress y se identificaron sus respectivos PIDs mediante comandos de monitoreo como ps aux y htop.

Posteriormente se utilizaron los comandos killall y pkill, que envían la señal SIGTERM, permitiendo que los procesos finalicen de manera ordenada. Después se generó nuevamente carga en el sistema y se aplicó la señal SIGKILL utilizando kill -9 sobre procesos específicos para forzar su finalización inmediata.

Finalmente se observó el estado de los procesos resultantes y la aparición de procesos Zombie dentro del sistema.

Análisis de resultados

La señal SIGTERM permitió que los procesos finalizaran de forma controlada, liberando adecuadamente los recursos utilizados. En contraste, la señal SIGKILL produjo una terminación inmediata gestionada por el kernel, sin permitir que los procesos realizaran tareas de limpieza previas a su cierre.

Durante las pruebas se observaron procesos en estado Zombie (z o <defunct>), los cuales corresponden a procesos que ya finalizaron su ejecución, pero cuyos registros aún permanecen en la tabla de procesos hasta que son recogidos por su proceso padre.

Según los registros obtenidos, llegaron a acumularse hasta diez procesos Zombie durante las diferentes pruebas realizadas, evidenciando la importancia de una correcta gestión de procesos hijos en aplicaciones y servicios.

Figura 24. PIDs stress, python3 y dd.

```
root@bbe283d77de4:/app# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.6 11481644 53264 ?        Ssl   21:10   0:00 node server.js
root        14  0.0  0.0   4712   3360 pts/0    Ss    21:11   0:00 bash
root        506  3.9  0.0      0      0 pts/0    Z     21:28   0:35 [stress] <defunct>
root        507  3.9  0.0      0      0 pts/0    Z     21:28   0:35 [stress] <defunct>
root       1200  0.0  0.0   3376   1760 pts/0    S     21:39   0:00 stress --cpu 2
root       1201  104  0.0   13840   7520 pts/0    R     21:39   4:18 python3 -c while True: pass
root       1202  104  0.0   3572   2400 pts/0    R     21:39   4:18 dd if=/dev/zero of=/dev/null bs=1M
root       1203  104  0.0   3376    320 pts/0    R     21:39   4:18 stress --cpu 2
root       1204  104  0.0   3376    320 pts/0    R     21:39   4:18 stress --cpu 2
root       1206  0.0  0.0   8108   4160 pts/0    R+    21:43   0:00 ps aux
```

Figura 25. SIGTERM (educado).

```
root@bbe283d77de4:/app# killall stress
[2] Terminated stress --cpu 2
root@bbe283d77de4:/app# pkill -f python3
root@bbe283d77de4:/app# pkill dd
[3]- Terminated python3 -c "while True: pass"
[4]+ Terminated dd if=/dev/zero of=/dev/null bs=1M
```

Figura 26. Htop para verificar.

```
0[ 0.7%] 3[ 0.0%] 6[ 0.0%] 9[ 0.0%] 10[ 0.0%] 12[ 0.0%] 16[ 0.0%] 19[ 0.7%]
1[ 0.0%] 4[ 0.0%] 7[ 0.0%] 11[ 0.0%] 14[ 0.0%] 17[ 0.0%]
2[ 0.0%] 5[ 0.0%] 8[ 0.0%] 13[ 0.7%] 15[ 0.0%] 18[ 0.0%]
Mem[|||||] 1.12G/7.57G Tasks: 7, 6 thr, 0 kthr, 1 running
Swp[ ] 0K/2.00G Load average: 0.54 1.48 1.10
Uptime: 00:41:31

Main T/O
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.41 node server.js
8 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 40160 S 0.0 0.7 0:00.00 node server.js
14 root 20 0 4712 3360 2560 S 0.0 0.0 0:00.07 bash
506 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
507 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
1203 root 20 0 0 0 0 Z 0.0 0.0 4:41.39 stress
1204 root 20 0 0 0 0 Z 0.0 0.0 4:41.38 stress
1210 root 20 0 5072 3680 2720 R 0.0 0.0 0:00.00 htop
```

Figura 27. SIGKILL.

```
root@bbe283d77de4:/app# killall stress
[2] Terminated stress --cpu 2
root@bbe283d77de4:/app# pkill -f python3
root@bbe283d77de4:/app# pkill dd
[3]- Terminated python3 -c "while True: pass"
[4]+ Terminated dd if=/dev/zero of=/dev/null bs=1M
root@bbe283d77de4:/app# htop
root@bbe283d77de4:/app# stress --cpu 2 &
python3 -c "while True: pass" &
[1] 1211
[2] 1212
root@bbe283d77de4:/app# stress: info: [1211] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Figura 28. Obtener los PIDs.

```
root@bbe283d77de4:/app# ps aux | grep -E "stress|python3"
root      506  2.9  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root      507  2.9  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root     1203 51.9  0.0      0      0 pts/0    Z   21:39   4:41 [stress] <defunct>
root     1204 51.9  0.0      0      0 pts/0    Z   21:39   4:41 [stress] <defunct>
root     1211  0.0  0.0    3376 1760 pts/0    S   21:48   0:00 stress --cpu 2
root     1212 104  0.0   13840 7680 pts/0    R   21:48   0:40 python3 -c while True: pass
root     1213 104  0.0    3376   160 pts/0    R   21:48   0:40 stress --cpu 2
root     1214 104  0.0    3376   160 pts/0    R   21:48   0:40 stress --cpu 2
root     1216  0.0  0.0    3332 1280 pts/0    R+  21:48   0:00 grep -E stress|python3
```

Figura 29. Matar Forzado.

```
root@bbe283d77de4:/app# ps aux | grep -E "stress|python3"
root      506  2.9  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root      507  2.9  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root     1203 51.9  0.0      0      0 pts/0    Z   21:39   4:41 [stress] <defunct>
root     1204 51.9  0.0      0      0 pts/0    Z   21:39   4:41 [stress] <defunct>
root     1211  0.0  0.0    3376 1760 pts/0    S   21:48   0:00 stress --cpu 2
root     1212 104  0.0   13840 7680 pts/0    R   21:48   0:40 python3 -c while True: pass
root     1213 104  0.0    3376   160 pts/0    R   21:48   0:40 stress --cpu 2
root     1214 104  0.0    3376   160 pts/0    R   21:48   0:40 stress --cpu 2
root     1216  0.0  0.0    3332 1280 pts/0    R+  21:48   0:00 grep -E stress|python3
root@bbe283d77de4:/app# kill -9 <PID_stress>
bash: syntax error near unexpected token 'newline'
root@bbe283d77de4:/app# # Mata forzado
kill -9 <PID_stress>
kill -9 <PID_python3>
bash: syntax error near unexpected token 'newline'
bash: syntax error near unexpected token 'newline'
root@bbe283d77de4:/app# kill -9 1211
kill -9 1213
kill -9 1214
kill -9 1212
root@bbe283d77de4:/app# ps aux | grep -E "stress|python3"
root      506  1.7  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root      507  1.7  0.0      0      0 pts/0    Z   21:29   0:35 [stress] <defunct>
root     1203 20.0  0.0      0      0 pts/0    Z   21:40   4:41 [stress] <defunct>
root     1204 20.0  0.0      0      0 pts/0    Z   21:40   4:41 [stress] <defunct>
root     1213 104  0.0      0      0 pts/0    Z   21:49  15:43 [stress] <defunct>
root     1214 104  0.0      0      0 pts/0    Z   21:49  15:43 [stress] <defunct>
root     1218  0.0  0.0    3332 1440 pts/0    S+  22:04   0:00 grep -E stress|python3
[1]-  Killed                  stress --cpu 2
[2]+  Killed                  python3 -c "while True: pass"
```

Figura 30. Verificación en el Htop.

```
0[ 0.0%] 3[ 0.0%] 6[ 0.7%] 9[ 0.0%] 10[ 0.0%] 13[ 0.0%] 16[ 0.0%] 19[ 0.0%]
1[ 0.0%] 4[ 0.0%] 7[ 0.0%] 11[ 0.0%] 14[ 0.0%] 17[ 0.0%]
2[ 0.7%] 5[ 0.0%] 8[ 0.0%] 12[ 0.0%] 15[ 0.7%] 18[ 0.0%]
Mem[|||||] 1.13G/7.57G Tasks: 9, 6 thr, 0 kthr; 1 running
Swap[|||||] 0K/2.00G Load average: 1.14 2.47 2.21
Uptime: 00:57:59

Main: 1/70
PID USER DRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.01 node server.js
6 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.00 node server.js
9 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.02 node server.js
10 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.01 node server.js
11 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.01 node server.js
12 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.02 node server.js
13 root 20 0 11.2G 53264 48160 S 0.0 0.7 0:00.00 node server.js
10 root 20 0 4712 3368 2560 S 0.0 0.0 0:00.07 bash
506 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
507 root 20 0 0 0 0 Z 0.0 0.0 0:35.62 stress
1203 root 20 0 0 0 0 Z 0.0 0.0 4:41.39 stress
1204 root 20 0 0 0 0 Z 0.0 0.0 4:41.38 stress
1213 root 20 0 0 0 0 Z 0.0 0.0 15:43:47 stress
1214 root 20 0 0 0 0 Z 0.0 0.0 15:43:49 stress
1219 root 20 0 5072 3840 2880 R 0.0 0.0 0:00.00 htop
```

B — Gestión de prioridades con nice y renice

Objetivo de la fase

Comprender cómo el scheduler del sistema operativo distribuye el tiempo de CPU entre procesos con diferentes niveles de prioridad.

Procedimiento realizado

Se ejecutaron procesos de carga utilizando distintos valores de prioridad mediante el comando `nice`. Posteriormente se modificó la prioridad de procesos ya existentes utilizando `renice`, permitiendo observar cambios dinámicos sin necesidad de reiniciar las aplicaciones.

Durante la ejecución se monitorearon las columnas PRI y NI en `htop`, verificando cómo el sistema asignaba recursos a cada proceso.

Tabla 5. Prioridades utilizadas durante la prueba

Comando	Valor NI	Prioridad
<code>nice -n 19 stress --cpu 1</code>	19	Baja
<code>nice -n -20 stress --cpu 1</code>	-20	Alta
<code>renice</code>	Variable	Modificada en ejecución

Análisis de resultados

Las pruebas demostraron que los procesos con valores NI más bajos reciben una mayor prioridad por parte del scheduler del sistema operativo. En consecuencia, dichos procesos obtuvieron más tiempo de CPU frente a aquellos configurados con prioridades menores.

Asimismo, el comando `renice` permitió modificar la prioridad de procesos ya activos sin necesidad de detenerlos, mostrando cómo Linux puede ajustar dinámicamente la distribución de recursos según las necesidades del sistema.

Las capturas obtenidas evidencian claramente la diferencia entre los valores NI de cada proceso y el impacto que estas prioridades tienen sobre el uso del procesador.

Figura 31. Dos stress con diferente prioridad.

Baja prioridad `nice -n 19 stress --cpu 2 &`

Alta prioridad `nice -n -20 stress --cpu 2 &`

```
root@bbe283d77de4:/app# #nice -n 19 stress --cpu 2 &
root@bbe283d77de4:/app# nice -n -20 stress --cpu 2 &
[1] 1220
root@bbe283d77de4:/app# nice: cannot set niceness: Permission denied
stress: info: [1220] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Figura 32. Observación en el Htop.

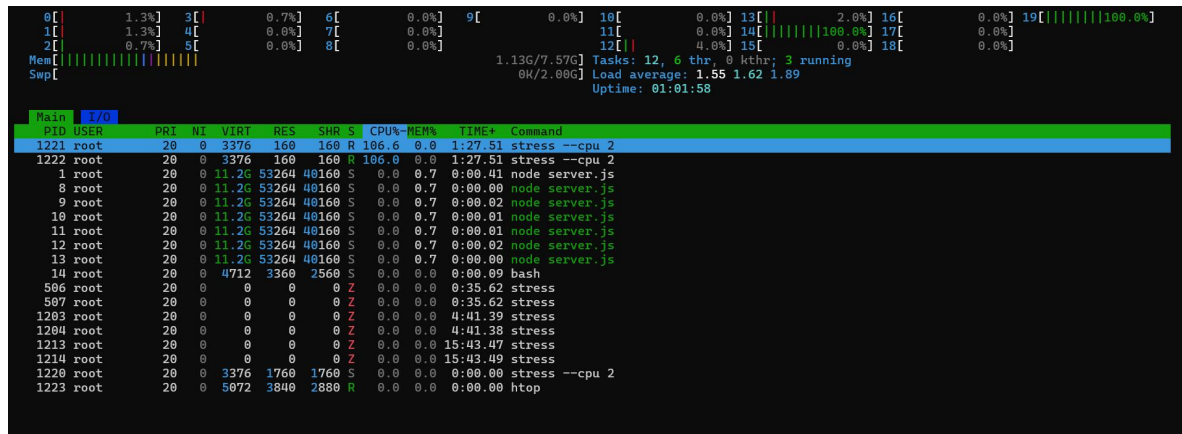


Figura 33. Cambio de prioridad de un proceso ya recorriendo.

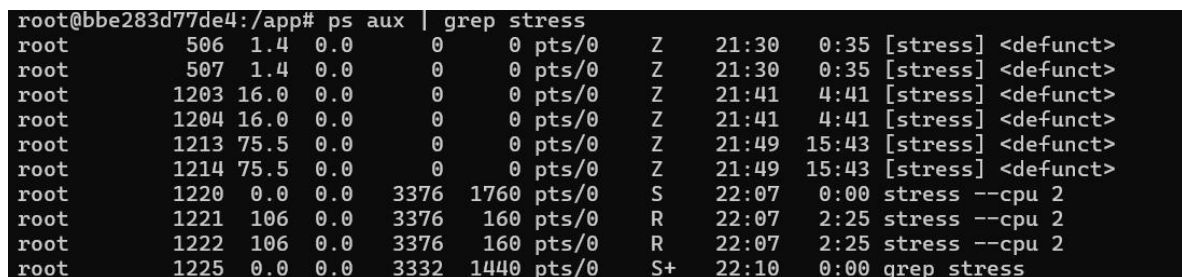


Figura 34 y 35. Observación en el Htop.

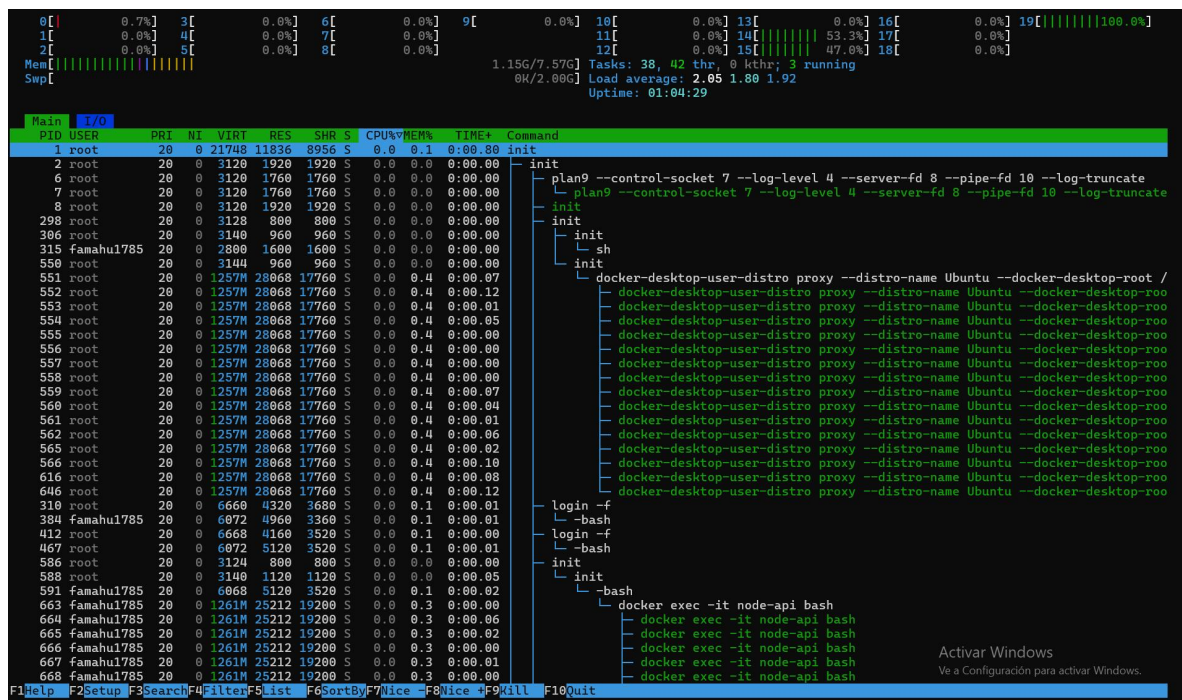


Tabla 6. Comparación antes y después de cpulimit

Estado	Consumo aproximado de CPU
Sin limitación	≈100%
Con cpulimit -l 30	≈30%

Análisis de resultados

A diferencia de las pruebas anteriores, en esta fase no se eliminó el proceso responsable de la carga. En su lugar, se redujo su consumo de CPU mediante una limitación controlada.

Los resultados mostraron que el proceso continuó ejecutándose normalmente, pero con un porcentaje de utilización considerablemente menor. Esto demuestra que herramientas como cpulimit pueden utilizarse para evitar que una aplicación monopolice los recursos del sistema sin necesidad de interrumpir su funcionamiento.

Al finalizar la prueba se eliminaron tanto el proceso de carga como el proceso encargado de aplicar la limitación, verificando mediante comandos de monitoreo que no quedaran instancias activas relacionadas con la prueba.

Figura 37. Lanza stress y anota PID.

```
root@bbe283d77de4:/app# stress --cpu 2 &
[1] 1228
root@bbe283d77de4:/app# stress: info: [1228] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
root@bbe283d77de4:/app# ps aux | grep stress | grep -v defunct | grep -v grep
root      1228  0.0  0.0  3376  1760 pts/0    S   22:14   0:00 stress --cpu 2
root      1229  105  0.0  3376   320 pts/0    R   22:14   0:19 stress --cpu 2
root      1230  105  0.0  3376   320 pts/0    R   22:14   0:19 stress --cpu 2
```

Figura 38. Limitar al 30% de CPU.

```
root@bbe283d77de4:/app# cpulimit -p 1228 -l 30 &
[2] 1236
root@bbe283d77de4:/app# Process 1228 detected
|
```

Figura 39. Observación del Htop.

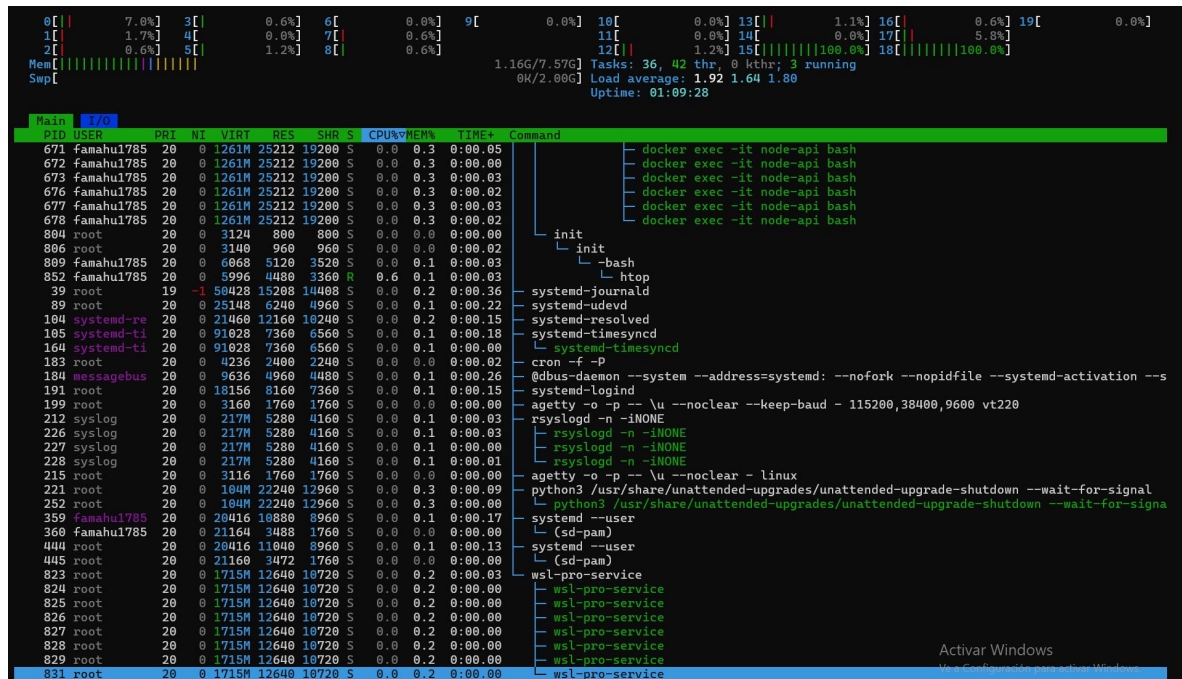


Figura 40. Limpieza final, Killall y el pkill.

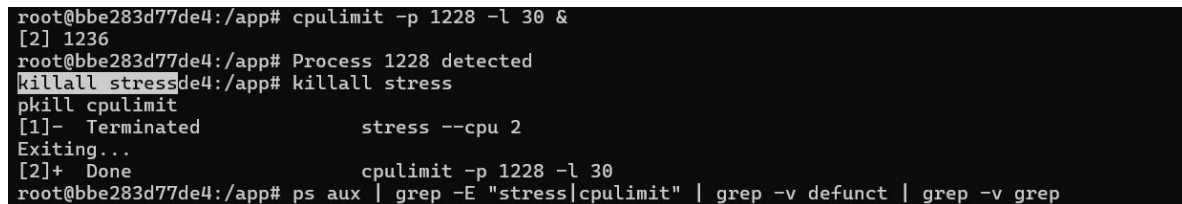
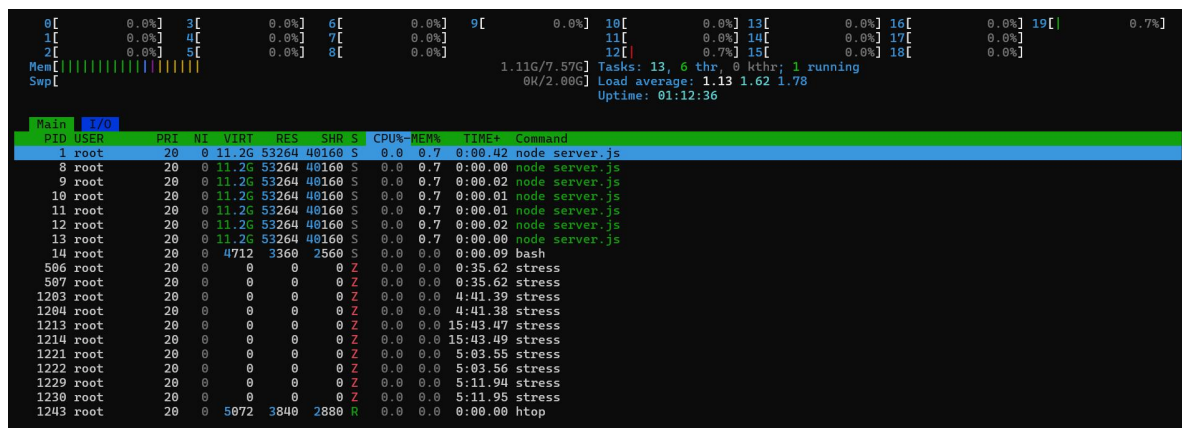


Figura 41. Observación final del Htop.



IV. Análisis y discusión de resultados

A. ¿Cuál es la diferencia entre kill y kill -9? ¿Cuándo usarías cada uno?

El comando kill envía la señal **SIGTERM**, la cual solicita al proceso que finalice de manera ordenada. Esto permite que el programa cierre archivos abiertos, guarde información pendiente y libere los recursos que está utilizando antes de terminar su ejecución.

Por otro lado, kill -9 envía la señal **SIGKILL**, que obliga al sistema operativo a finalizar el proceso de forma inmediata. En este caso, el proceso no tiene la oportunidad de realizar ninguna tarea de cierre, por lo que su terminación es completamente forzada.

Durante la práctica se observó que algunos procesos finalizaban correctamente utilizando killall stress, mientras que con kill -9 la terminación ocurría instantáneamente.

Generalmente se recomienda utilizar primero kill, ya que es una forma más segura de finalizar procesos. El uso de kill -9 debe reservarse para situaciones en las que el proceso no responde o no puede cerrarse de manera normal.

B. ¿Qué pasó con el proceso de mayor prioridad frente al de menor prioridad durante la carga?

En esta práctica se utilizaron diferentes niveles de prioridad mediante el comando nice. Se observó que el proceso con una prioridad más alta recibió una mayor cantidad de tiempo de CPU, mientras que el proceso con prioridad más baja tuvo menos oportunidades de ejecución.

También se evidenció que asignar prioridades muy altas, como nice -20, requiere permisos de administrador. Al intentar realizar esta acción dentro del contenedor Docker apareció un mensaje de error indicando falta de permisos.

Este mecanismo permite que el sistema operativo distribuya los recursos de manera más eficiente, favoreciendo los procesos más importantes cuando existe competencia por el uso del procesador.

C. ¿En qué situaciones sería preferible bajar la prioridad de un proceso en lugar de finalizarlo?

Reducir la prioridad de un proceso puede ser una mejor alternativa que finalizarlo cuando este continúa realizando una tarea importante, pero no requiere atención inmediata.

Por ejemplo, si un proceso está ejecutando una operación larga y consume una gran cantidad de recursos, finalizarlo implicaría perder el trabajo realizado hasta ese momento. En cambio, al disminuir su prioridad mediante `renice`, el proceso continúa funcionando, pero permite que otras aplicaciones tengan acceso preferencial al procesador.

Durante el laboratorio se observó este comportamiento al modificar la prioridad de procesos en ejecución, comprobando que seguían activos mientras reducían su impacto sobre el rendimiento general del sistema.

D. ¿Qué indica la columna NI en htop? ¿Cuál es su rango?

La columna **NI (Niceness)** muestra el nivel de prioridad asignado a un proceso para el uso de la CPU. Este valor es utilizado por el planificador del sistema operativo para decidir cuánto tiempo de procesamiento recibe cada proceso.

El rango de valores va desde **-20 hasta 19**. Los valores negativos representan una prioridad más alta, mientras que los valores positivos indican una prioridad más baja.

Durante la práctica se comprobó que únicamente los usuarios con privilegios administrativos pueden asignar valores negativos, ya que al intentar establecer una prioridad de -20 se generó un mensaje de falta de permisos.

E. ¿Qué es un proceso zombi y por qué es un problema?

Un proceso zombi es un proceso que ya terminó su ejecución, pero cuya información permanece registrada en la tabla de procesos porque el proceso padre aún no ha recogido su estado de finalización.

Estos procesos pueden identificarse en herramientas como `htop` mediante el estado **Z**, o en `ps aux` con la etiqueta **<defunct>**. Durante la práctica se observaron varios procesos en este estado después de finalizar cargas de trabajo.

Aunque los procesos zombis no consumen CPU y utilizan muy poca memoria, siguen ocupando entradas en la tabla de procesos del sistema. Si se acumulan en grandes cantidades, pueden agotar los identificadores de proceso disponibles y afectar el funcionamiento normal del sistema.

Por esta razón, es importante que las aplicaciones administren correctamente sus procesos hijos y liberen los recursos asociados cuando estos finalizan.

VI. CONCLUSIONES

- Se comprobó que la herramienta stress permite generar cargas controladas sobre CPU y memoria, facilitando el análisis del comportamiento del sistema bajo diferentes niveles de exigencia.
- El uso de herramientas de monitoreo como htop, free, ps y pstree permitió identificar en tiempo real el consumo de recursos y la interacción entre procesos.
- Las pruebas con kill, nice, renice y cpublimit demostraron diferentes mecanismos de administración de procesos, prioridades y control de recursos disponibles en Linux.
- Se observó la aparición de procesos zombis durante algunas pruebas, evidenciando la importancia de una correcta gestión de procesos hijos y señales de terminación.
- El laboratorio permitió comprender de manera práctica cómo Linux distribuye y controla los recursos del sistema para mantener la estabilidad y el rendimiento bajo condiciones de carga.

REFERENCIAS

[1] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA, USA: No Starch Press, 2010.

[2] W. Shotts, *The Linux Command Line*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2019.

[3] [Docker Documentation](#)

[4] [Linux man-pages Project](#)

[5] [GNU Core Utilities Documentation](#)